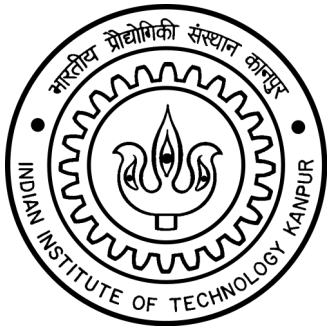


Indian Institute of Technology Kanpur

Department of Mechanical Engineering

Project Jafar

Design and Development of a 3D-Printed Differential-Actuated Prosthetic Arm

---

Course: ME252 — Theory of Mechanisms and Machines

Project Team

Anandan Iyer	anandani24@iitk.ac.in	**Prakhar**	prakhark24@iitk.ac.in
Ayush Dwivedi	ayushd24@iitk.ac.in	**Sarthak Patel**	sarthakp24@iitk.ac.in
Dipon Konar	diponk24@iitk.ac.in	**Shubham**	shubham24@iitk.ac.in
Lakshya Gupta	lakshyag24@iitk.ac.in		

June 16, 2026

Contents

1 Overview 1

2 Iteration 1: Proof-of-Concept Prototype 1

2.1 Mechanical Design . . . 1

2.2 Control System . . . 1

3 Iteration 2: Differential-Actuated Multi-DOF System 1

3.1 Redesign Rationale . . . 1

3.2 Three-DOF Mechanical Architecture . . . 1

3.3 Differential Finger Actuation . . . 2

3.3.1 Objective . . . 2

3.3.2 Working principle . . . 2

3.3.3 Implementation . . . 2

3.4 CAD Redesign . . . 3

3.5 Electronics and Control System . . . 3

3.5.1 Control Architecture . . . 3

3.5.2 Wireless Control Methodologies . . . 4

4 Results and Conclusion 4

5 Future Scope 4

References 5

A Appendix 5

A.1 Python Voice Recognition Script . . . 5

A.2 ESP32 Control Code . . . 6

1 Overview

Project Jafar is a noble effort to design and develop a prosthetic arm within an affordable price range for an 8 year old kid named Jafar who doesn't have the ability to use either of his arms. The first iteration involved fabricating a singe-DOF proof-of-concept which incorporated finger-actuation and the ability to grip fragile objects. In the second iteration, we worked on developing a **three-DOF platform** incorporating finger actuation and, elbow flexo-extension driven by a **differential actuation mechanism** and controlled wirelessly via an ESP32 microprocessor.

Parameter	Iteration 1	Iteration 2
DOF	1 (Finger actuation)	3 (Finger, Elbow, Prono)
Actuation	Direct string, single motor	Differential mechanism
Microcontroller	Arduino Nano	ESP32
Input modality	EMG sensor	Bluetooth & Wi-Fi voice
Tendon routing	Ad-hoc, unguided	Guided channels + hooks

Table 1: High-level comparison across design iterations.

2 Iteration 1: Proof-of-Concept Prototype

2.1 Mechanical Design

Iteration 1 employed a **single motor** which drives fingers simultaneously through a combined string tendon arrangement: individual finger strings were merged upstream and connected to a single motor. Although the mechanism was quick to implement, it was prone to multiple failures.

- **No force distribution:** equal motor tension was applied uniformly to all tendons regardless of per-finger contact resistance or object interaction which resulted in disproportionate actuation and instability.
- **Inconsistent string tension:** unguided routing introduced variable friction, slack, and path-dependent losses, which lead to unpredictable grip profiles.
- **Recurring string failures:** uncontrolled routing concentrated stress at sharp bends, contact edges, and termination points, accelerating wear and leading to frequent tendon breakage under cyclic loading.
- **Rigid inter-finger coupling:** all fingers were mechanically linked which prevented them from moving independently that highlighted another design flaw.

The elbow joint was kept passive for this prototype, which housed the 7.4V battery, which was used to drive the motor.

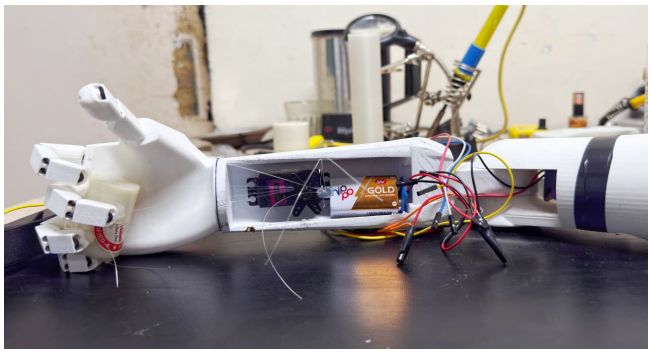


Figure 1: Iteration I prototype: signle-DOF combined-string finger actuation

2.2 Control System

The control architecture used an **Arduino Nano** to sample the analogue voltage from a surface **EMG sensor band**. Myoelectric potentials were kept as threshold to generate binary motor on/off commands. Reliable electrode placement was another hurdle; we were seldom able to position EMG bands correctly, which led to difficulties in interpreting the signal. Attempts were made to set the motor's position based on the signal strength received, but the attempts failed because the signal was not stable enough, this led to the limitation to keep fingers either fully open or closed, and not in any intermediate state.
These issues collectively led to the full architectural redesign leading to **Iteration 2.**

3 Iteration 2: Differential-Actuated Multi-DOF System

3.1 Redesign Rationale

The transition to Iteration 2 was driven by a few principal deficiencies, along with a shift in the control strategy from the EMG-based sensing used in the first iteration to a more reliable and implementable approach: **(i)** Structural inconsistencies in the CAD model with no defined tendon pathways or dedicated electronics accommodation. **(ii)** Recurring string failures from uncontrolled stress concentrations. **(iii)** Unreliable finger control from a coupled tendon architecture. **(iv)** The single-DOF constraint precluding the three-DOF mobility required for functional prosthetic use.

3.2 Three-DOF Mechanical Architecture

The upgraded system implements three independently motorised degrees of freedom. **DOF 1** provides finger actuation via the differential mechanism (Section 3.3). **DOF 2** delivers elbow flexo-extension through a direct-drive scheme in which the motor housing is fixed to the upper-arm segment and the output shaft is rigidly coupled to the forearm, producing pure joint rotation without additional transmission elements. **DOF 3** enables forearm pronosupination via a motor whose axis is co-axial with the forearm's longitudinal centre-line. All three motors are independently addressable through dedicated PWM channels on the ESP32.

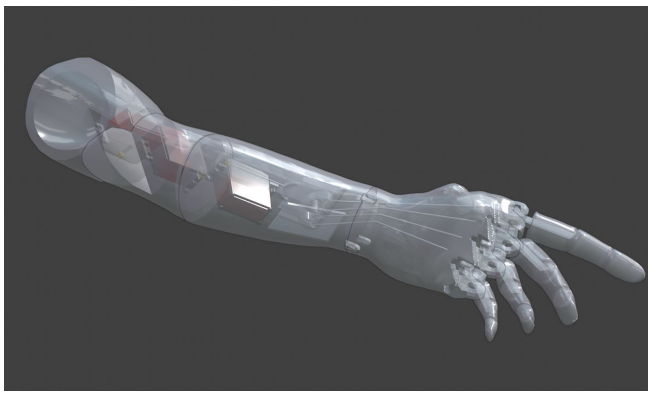


Figure 2: Iteration 2 three-DOF kinematic layout: motor placement and joint axes.

3.3 Differential Finger Actuation

3.3.1 Objective

As established in Section 2.1 (Mechanical Design), the initial finger actuation required a more robust system for force distribution, hence we will be implementing a differential mechanism for this part of the prosthetic. The differential mechanism enabled a balanced and adaptive force distribution among the fingers, allowing actuation to adjust based on resistance at each finger. If one of the fingers faces resistance when gripping a particular object, the rest of the the fingers continue to grip around the object, and when all the fingers curl, further tension in the strings start to grip the object tighter. This resulted in a smoother motion and improved grasping performance, especially when contact conditions vary across fingers.

3.3.2 Working principle

Based on the kinematic constraints of strings, the displacement of the primary pulley P_3 (is distributed through the differential tree according to the following relations:

$$D_5 + D_6 = 2D_7$$
$$D_1 + D_2 = 2D_5$$
$$D_3 + D_4 = 2D_6$$

When one of the fingers start feeling resistance, say finger 1 for example, it would stop moving, and rest of the motion would go instead to its coupled finger, in this case finger 2 ($D_1 = 0$ and $D_2 = 2D_5$). In case both the coupled fingers feel resistance, then the motion would instead go to the other coupled pulley. (In this case for example, P_1 stops moving and P_2 take all the motion). The physical definition of each variable is illustrated in Figure 3:

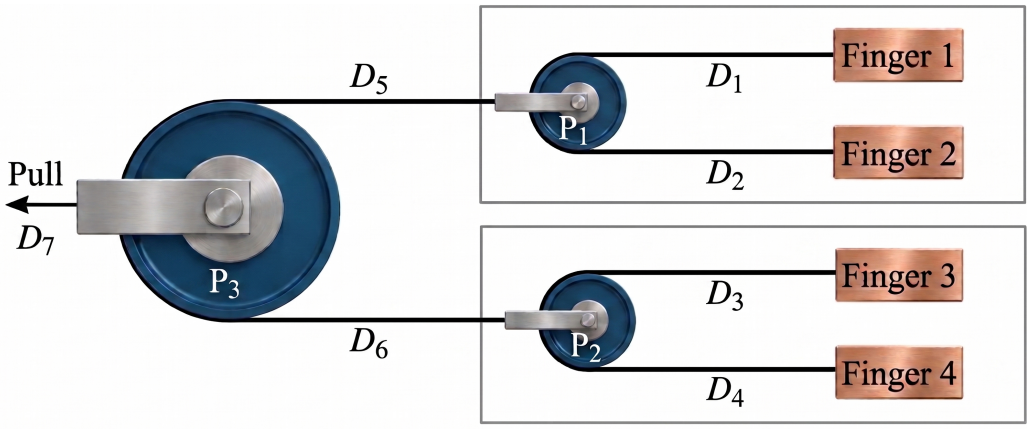
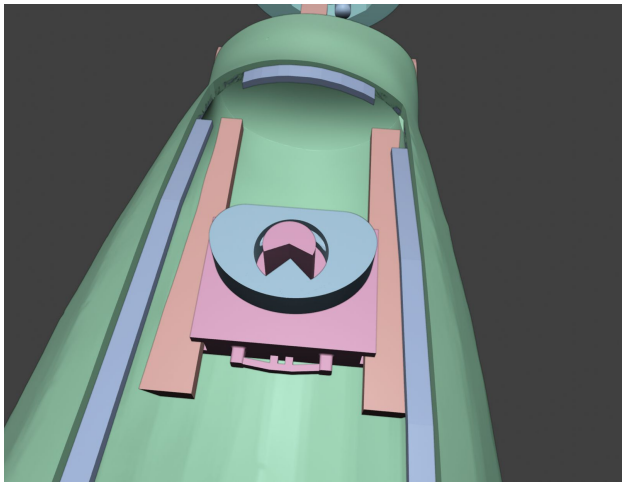


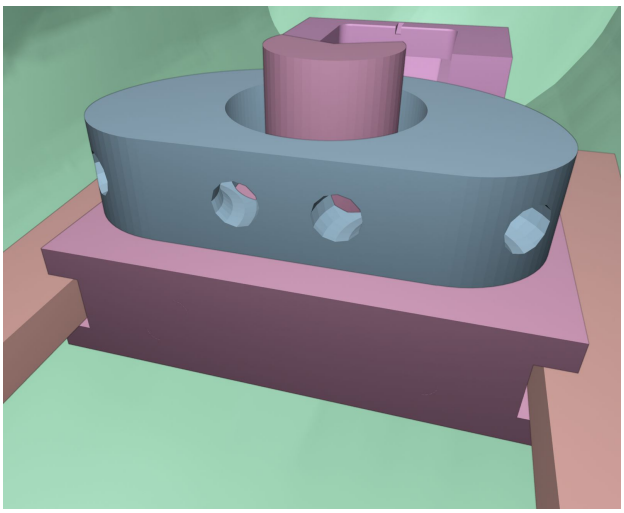
Figure 3: Simplified view of a differential mechanism

3.3.3 Implementation

In our implementation of the differential mechanism, given space constraints, we had to move away from the basic pulley-based Atwood machine shown in Figure 3. Our approach combined the pulleys P_1, P_2, and P_3 into one single mechanism. The heart of the mechanism is shown in blue atop the pink slide, which slides comfortably into the red slot (as seen in Figure 4). In the front part, which opens towards the fingers (which functionally acts like P_1 and P_2 and are similar to traditional pulleys), there are holes for strings to enter, with a toroidal cavity for each pair of fingers. The string can move freely (after using a suitable lubricant, so the string does not wear out). To implement the P_3, the Blue part uses the red part as a hinge to rotate. When there is unequal motion between pulleys P_1 and P_2, the blue part rotates about the red part; thus, if required and one of the pulleys stalls due to resistance, the mechanism can continue sliding, on the tracks, with the blue part rotating to accomodate the movement.



(a) Bird's-eye view.



(b) Close-up view.

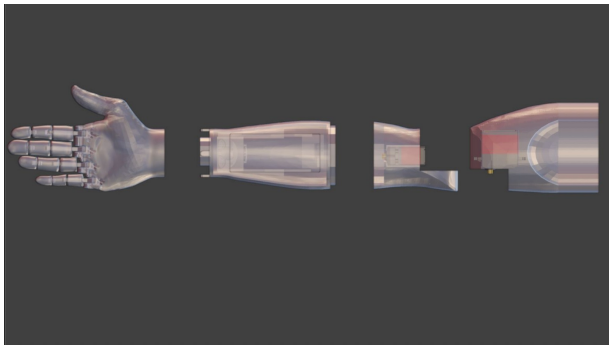
Figure 4: Views of the differential mechanism assembly.

Property	Iteration 1 - Prototyping	Iteration 2 - Differential
Force distribution	Equal, load-independent	Adaptive per-finger
Grasp adaptability	None (rigid coupling)	Passive compliance
Tendon routing	Unguided, ad-hoc	Guided channels + hooks
Frictional losses	High (uncontrolled contacts)	Reduced (smooth guided paths with lubrication)

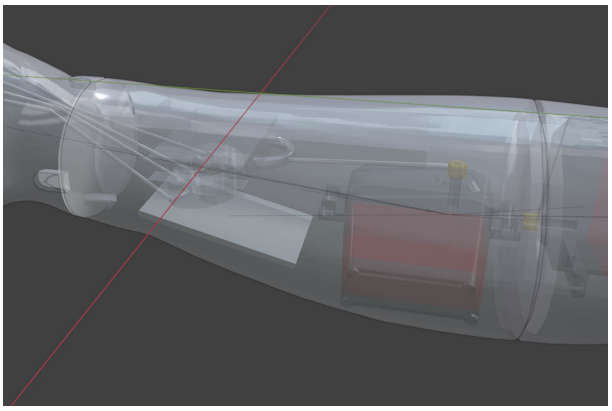
Table 2: Actuation mechanism comparison across iterations.

3.4 CAD Redesign

The updated design adopts a more structured and integrated approach to improve overall system performance. The forearm geometry was redesigned to enable functioning of both the actuation system and electronics within a dedicated internal cavity which significantly improves the spatial organization, component protection, and the ease of assembly. Carefully planned provisions for servo placement and wire routing have been useful in reducing clutter, minimized interference between components, and enhanced long-term maintainability and serviceability. The forearm was also modified to better integrate the differential mechanism which resulted in more efficient force transmission and smoother, more coordinated motion during operation. In addition, the upper arm was redesigned to incorporate an extra degree of freedom at the elbow joint(passive in the initial iteration), which increases the overall range of motion of the system. Further improvements were made to the palm and finger assemblies. Guided string channels were built directly into the CAD model, replacing the earlier ad-hoc routing approach to ensure a cleaner and more reliable tendon path. Intermediate hooks and supports were added along the tendon path to maintain proper alignment to reduce slack and minimize frictional losses during the motion. These changes ensure a more consistent tension distribution and smoother, more precise actuation across all fingers.



(a) Iteration 1: unstructured interior



(b) Iteration 2: CAD with integrated cavity, servo mounts, and wire channels

3.5 Electronics and Control System

3.5.1 Control Architecture

The control system is built around the **ESP32** microcontroller, selected for its integrated Bluetooth and Wi-Fi capabilities. This enables multiple wireless input methodologies on a single hardware platform while providing sufficient PWM-capable GPIO pins for independent control of the finger and elbow actuation motors.

To ensure stable operation, the motor supply and logic supply are regulated independently. This minimizes switching transients generated by the motors, thereby improving system reliability and preventing voltage fluctuations from affecting control performance.

3.5.2 Wireless Control Methodologies

During development, we tried out two wireless approaches. This revealed a clear trade-off: one was much simpler to implement, but the other gave the system far more autonomy.

Bluetooth Serial Interface

In the initial approach, the ESP32 was configured as a Bluetooth device and paired with a mobile phone. Commands were transmitted using a *Bluetooth Serial Terminal* application, with **Google Assistant** acting as a voice-to-text front-end.
The spoken command was converted to text via cloud-based processing and relayed to the ESP32, where it was parsed against predefined command tokens. While straightforward to implement, this method suffered from internet dependency, increased latency, and reduced reliability under wireless interference.

Wi-Fi Offline Voice Interface

To overcome these limitations, a Wi-Fi-based offline voice interface was implemented. A Python script running on a laptop connected to the ESP32 over a local Wi-Fi network utilizes a **pretrained offline speech recognition library (Vosk)** to detect predefined command phrases. Recognized commands are transmitted to the ESP32 via HTTP requests, enabling real-time actuation without reliance on internet connectivity. The command structure was designed for clarity and robustness. Finger actuation (DOF 1) is controlled using discrete commands, while higher degrees of freedom corresponding to elbow and rotational motion (DOF 2 and DOF 3) follow a structured format. The actuation angles are limited to fixed values such as 0°, 30°, 60°, and 90° for easier control and can be further increased. This separation simplifies parsing and improves recognition reliability.

Voice Command	DOF Target	Motor Action
`grip`	DOF 1 (Fingers)	Close fingers via differential actuation
`loose`	DOF 1 (Fingers)	Open fingers (release motion)
`motor N turn` θ	DOF 2 / DOF 3	Rotate motor N to θ°

Table 3: Voice command vocabulary and corresponding motor actions.

The Wi-Fi-based interface provides a more reliable and low-latency control solution compared to the Bluetooth approach, making it better suited for real-time prosthetic operation.

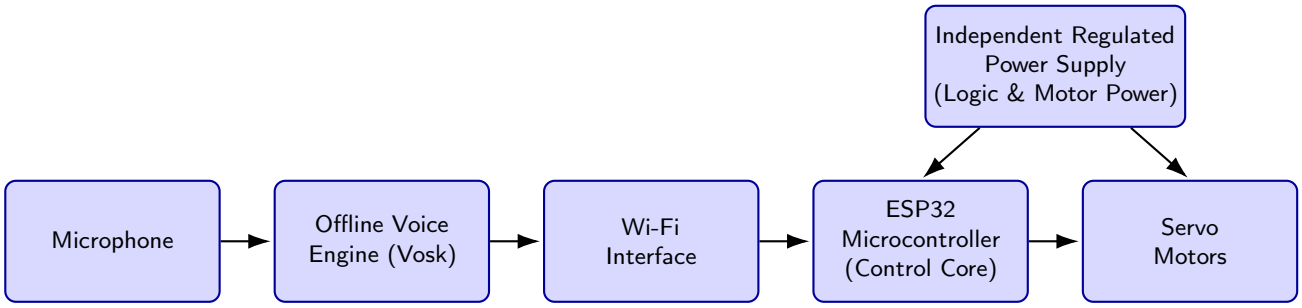


Figure 5: System control architecture and offline voice communication flow.

4 Results and Conclusion

The project progressed through two iterations, each improving upon the limitations of the previous design.
Iteration 1 resulted in a fabricated and tested, successfully demonstrating basic finger actuation. However, issues such as inconsistent tension and unreliable control highlighted the need for redesign.
In **Iteration 2**, we focused on improving different parts of the system separately. The updated CAD design made the structure more organized and helped in proper routing of the strings. The differential mechanism showed better force distribution and smoother finger movement during testing. On the electronics side, the ESP32 control system and the Wi-Fi based voice interface were tested independently and worked reliably with low delay.
At this stage, all the major subsystems have been developed and tested individually. The next step is to integrate everything together and test the full system with all degrees of freedom working simultaneously. Overall, the second iteration provides a much more stable and improved base compared to the first prototype.

5 Future Scope

The next step is to integrate all the subsystems and test the complete prosthetic arm. This includes running all three degrees of freedom together and checking performance during actual grasping tasks. The control system can be improved by bringing back **EMG-based input** or using **physical buttons** along with voice commands for more intuitive use. Adding **feedback systems**, such as force or position sensing, can help improve accuracy and control. On the mechanical side, further refinement of the differential mechanism and string routing can reduce friction and improve durability. Reducing weight and making the design more compact will also improve usability. The command input can be expanded to allow smoother and more precise movements instead of only fixed angle steps. Overall, the current design provides a good base for developing a more reliable and user-friendly prosthetic arm.

References

V. Khatik and A. Saxena, "A Novel Slack Tolerant, Sliding Pulley Differential Mechanism for Adaptive Grasping," *Proc. ASME IDETC-CIE, 47th Mechanisms and Robotics Conf.*, Vol. 8, p. V008T08A071, Aug. 2023. doi: 10.1115/DETC2023-115115.

Viral Science Creativity, "Arduino Flex Sensor Controlled Robot Hand," Sep. 18, 2022. Available: `https://www.viralsciencecreativity.com/post/arduino-flex-sensor-controlled-robot-hand`.

Enabling the Future, "Upper Limb Prosthetics," Available: `https://enablingthefuture.org/upper-limb-prosthetics/`

Kwawu Arm Project, "Kwawu Arm 3.0 Wrap Version," Available: `https://www.printables.com/model/653545-kwawu-arm-30-wrap-version`

A Appendix

A.1 Python Voice Recognition Script

```
import sounddevice as sd
import json
from vosk import Model, KaldiRecognizer
import requests

model = Model("model")

rec = KaldiRecognizer(
    model, 16000,
    '["grip","loose","motor one","motor two","turn","zero","thirty","sixty","ninety"]'
)

ESP_IP = "192.168.4.1"

motor_map = {
    "one": 1,
    "two": 2
}

angle_map = {
    "zero": 0,
    "thirty": 30,
    "sixty": 60,
    "ninety": 90
}

def send(cmd, motor=None, angle=None):
    try:
        if cmd in ["grip", "loose"]:
            url = f"http://{ESP_IP}/cmd?mode={cmd}"
        else:
            url = f"http://{ESP_IP}/cmd?mode=move&motor={motor}&angle={angle}"

        requests.get(url)
        print("Sent:", url)

    except:
        print("ESP not connected")

def process(text):
    print("Heard:", text)

    if "grip" in text:
        send("grip")
        return

    if "loose" in text:
        send("loose")
        return

    motor = None
```

```
    angle = None

    for word in motor_map:
        if word in text:
            motor = motor_map[word]

    for key in angle_map:
        if key in text:
            angle = angle_map[key]

    if motor and angle is not None:
        send("move", motor, angle)

def callback(indata, frames, time, status):
    if rec.AcceptWaveform(bytes(indata)):
        result = json.loads(rec.Result())
        text = result.get("text", "")

        if text:
            process(text)

print(" Speak now...")

with sd.RawInputStream(samplerate=16000,
                       blocksize=8000,
                       dtype='int16',
                       channels=1,
                       callback=callback):
    while True:
        pass
```

A.2 ESP32 Control Code

```

#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

WebServer server(80);

Servo finger;   // DOF 1
Servo elbow1;   // DOF 2
Servo elbow2;   // DOF 3

void handleCmd() {
  String mode = server.arg("mode");

  if (mode == "grip") {
    finger.write(90);  // close
  }

  else if (mode == "loose") {
    finger.write(0);   // open
  }

  else if (mode == "move") {
    int motor = server.arg("motor").toInt();
    int angle = server.arg("angle").toInt();

    if (motor == 1) elbow1.write(angle);
    if (motor == 2) elbow2.write(angle);
  }

  server.send(200, "text/plain", "OK");
}

void setup() {
```

```
  WiFi.softAP("ESP32_ARM", "12345678");

  finger.attach(13);
  elbow1.attach(12);
  elbow2.attach(14);

  server.on("/cmd", handleCmd);
  server.begin();
}

void loop() {
  server.handleClient();
}
```